Btrfs Filesystem Documentation

Btrfs

From **Btrfs Wiki**:

> Btrfs is a modern copy on write (CoW) filesystem for Linux aimed at implementing advanced features while also focusing on fault tolerance, repair and easy administration. Jointly developed at multiple companies, Btrfs is licensed under the GPL and open for contribution from anyone.

Warning: Btrfs has some features that are unstable. See the Btrfs Wiki's **Status**, **Is Btrfs stable?** and **Getting started** for more detailed information. See the **#Known issues** section.

Contents [hide]

Preparation

For user space utilities **install** the **`btrfs-progs`** package, which is required for basic operations.

If you need to boot from a Btrfs file system (i.e., your kernel and initramfs reside on a Btrfs partition), check if your **boot loader** supports Btrfs.

File system creation

The following shows how to create a new Btrfs **file system**. To convert an ext3/4 partition to Btrfs, see **#Ext3/4 to Btrfs conversion**. To use a partitionless setup, see **#Partitionless Btrfs disk**.

See **`mkfs.btrfs(8)`** for more information.

File system on a single device

To create a Btrfs filesystem on partition `/dev/partition`:

```
# mkfs.btrfs -L mylabel /dev/partition
```

The Btrfs default nodesize for metadata is 16KB, while the default sectorsize for data is equal to page size and autodetected. To use a larger nodesize for metadata (must be a multiple of sectorsize, up to 64KB is allowed), specify a value for the `nodesize` via the `-n` switch as shown in this example using 32KB blocks:

```
# mkfs.btrfs -L mylabel -n 32k /dev/partition
```

Note: According to `mkfs.btrfs(8) § OPTIONS`, "[a] smaller node size increases fragmentation but leads to taller b-trees which in turn leads to lower locking contention. Higher node sizes give better packing and less fragmentation at the cost of more expensive memory operations while updating the metadata blocks".

Multi-device file system

Warning: The RAID 5 and RAID 6 modes of Btrfs are fatally flawed, and should not be used for "anything but testing with throw-away data." **List of known problems and partial workarounds.** See **the Btrfs page on RAID5 and RAID6** for status updates (seems to not be updated).

Multiple devices can be used to create a RAID. Supported RAID levels include RAID 0, RAID 1, RAID 10, RAID 5 and RAID 6. Starting from kernel 5.5 RAID1c3 and RAID1c4 for 3- and 4- copies of RAID 1 level. The RAID levels can be configured separately for data and metadata using the `-d` and `-m` options respectively. By default the data is striped (`raid0`) and the metadata is mirrored (`raid1`). See **Using Btrfs with Multiple Devices** for more information about how to create a Btrfs RAID volume.

```
# mkfs.btrfs -d raid0 -m raid1 /dev/part1 /dev/part2 ...
```

You can create a **JBOD configuration**, where disks are seen as one filesystem, but files are not duplicated, using `-d single -m raid1` .

You must include either the `udev` hook or the `btrfs` hook in `/etc/mkinitcpio.conf` in order to use multiple Btrfs devices in a pool. See the **Mkinitcpio#Common hooks** article for more information.

Note:

- It is possible to add devices to a multiple-device filesystem later on. See the **Btrfs wiki article** for more information.
- Devices can be of different sizes. However, if one drive in a RAID configuration is bigger than the others, this extra space will not be used.
- Some **boot loaders** such as **Syslinux** do not support multi-device file systems.
- Btrfs does not automatically read from the fastest device, so mixing different kinds of disks results in inconsistent performance. See **this Stack Overflow answer** for details.

See **#RAID** for advice on maintenance specific to multi-device Btrfs file systems.

Configuring the file system

Copy-on-Write (CoW)

By default, Btrfs uses **copy-on-write** for all files all the time. See the **Btrfs Sysadmin Guide section** for implementation details, as well as advantages and disadvantages.

Disabling CoW

To disable copy-on-write for newly created files in a mounted subvolume, use the `nodatacow` mount option. This will only affect newly created files. Copy-on-write will still happen for existing files. The `nodatacow` option also disables compression. See `btrfs(5)` for details.

Note: From `btrfs(5) § MOUNT OPTIONS`: "within a single file system, it is not possible to mount some subvolumes with `nodatacow` and others with `datacow` . The mount option of the first mounted subvolume applies to any other subvolumes."

To disable copy-on-write for single files/directories do:

```
$ chattr +C /dir/file
```

This will disable copy-on-write for those operation in which there is only one reference to the file. If there is more than one reference (e.g. through `cp --reflink=always` or because of a filesystem snapshot), copy-on-write still occurs.

Note: From chattr man page: "For btrfs, the 'C' flag should be set on new or empty files. If it is set on a file which already has data blocks, it is undefined when the blocks assigned to the file will be fully stable. If the 'C' flag is set on a directory, it will have no effect on the directory, but new files created in that directory will have the No_COW attribute."

Tip: In accordance with the note above, you can use the following trick to disable copy-on-write on existing files in a directory:

```
$ mv /path/to/dir /path/to/dir_old
$ mkdir /path/to/dir
$ chattr +C /path/to/dir
$ cp -a /path/to/dir_old/* /path/to/dir
$ rm -rf /path/to/dir_old
```

Make sure that the data are not used during this process. Also note that `mv` or `cp --reflink` as described below will not work.

Creating lightweight copies

By default, when copying files on a Btrfs filesystem with `cp`, actual copies are created. To create a lightweight copy referencing to the original data, use the *reflink* option:

```
$ cp --reflink source dest
```

See the man page on **cp(1)** for more details on the `--reflink` flag.

Compression

Btrfs supports **transparent and automatic compression**. This reduces the size of files as well as significantly increases the lifespan of flash-based media by reducing write amplification. **[1][2][3]** It can also **improve performance**, in some cases (e.g. single thread with heavy file I/O), while obviously harming performance in other cases (e.g. multi-threaded and/or CPU intensive tasks with large file I/O). Better performance is generally achieved with the fastest compress algorithms, *zstd* and *lzo*, and some **benchmarks** provide detailed comparisons.

The `compress=alg` mount option enables automatically considering every file for compression, where `alg` is either `zlib`, `lzo`, `zstd`, or `no` (for no compression). Using this option, btrfs will check if compressing the first portion of the data shrinks it. If it does, the entire write to that file will be compressed. If it does not, none of it is compressed. With this option, if the first portion of the write does not shrink, no compression will be applied to the write even if the rest of the data would shrink tremendously. **[4]** This is done to prevent making the disk wait to start writing until all of the data to be written is fully given to btrfs and compressed.

The `compress-force=alg` mount option can be used instead, which makes btrfs skip checking if compression shrinks the first portion, and enables automatic compression for every file. In a worst-case scenario, this can cause (slightly) more space to be used, and CPU usage for no purpose. However, empirical testing on multiple mixed-use systems showed a

significant improvement of about 10% disk compression from using `compress-force=zstd` over just `compress=zstd` (which also had 10% disk compression), resulting in a total effective disk space saving of 20%.

Only files created or modified after the mount option is added will be compressed.

To apply compression to existing files, use the `btrfs filesystem defragment -calg` command, where `alg` is either `zlib`, `lzo` or `zstd`. For example, in order to re-compress the whole file system with **zstd**, run the following command:

```
# btrfs filesystem defragment -r -v -czstd /
```

To enable compression when installing Arch to an empty Btrfs partition, use the `compress` option when **mounting** the file system: `mount -o compress=zstd /dev/sdxY /mnt/`. During configuration, add `compress=zstd` to the mount options of the root file system in **fstab**.

Tip: Compression can also be enabled per-file without using the `compress` mount option; to do so apply `chattr +c` to the file. When applied to directories, it will cause new files to be automatically compressed as they come.

Warning:

- Systems using older kernels or **btrfs-progs** without `zstd` support may be unable to read or repair your filesystem if you use this option.
- **GRUB** introduced *zstd* support in 2.04. Make sure you have actually upgraded the bootloader installed in your MBR/ESP since then, by running `grub-install` with the appropriate options for your BIOS/UEFI setup, since that is not done automatically. See **FS#63235**.

View compression types and ratios

compsize takes a list of files (or an entire btrfs filesystem) and measures compression types used and effective compression ratios. Uncompressed size may not match the number given by other programs such as `du`, because every extent is counted once, even if it is reflinked several times, and even if part of it is no longer used anywhere but has not been garbage collected. The `-x` option keeps it on a single filesystem, which is useful in situations like `compsize -x /` to avoid it from attempting to look in non-btrfs subdirectories and fail the entire run.

Subvolumes

"A btrfs subvolume is not a block device (and cannot be treated as one) instead, a btrfs subvolume can be thought of as a POSIX file namespace. This namespace can be accessed via the top-level subvolume of the filesystem, or it can be mounted in its own right." **[5]**

Each Btrfs file system has a top-level subvolume with ID 5. It can be mounted as `/` (by default), or another subvolume can be **mounted** instead. Subvolumes can be moved around in the filesystem and are rather identified by their id than their path.

See the following links for more details:

- **Btrfs Wiki SysadminGuide#Subvolumes**
- **Btrfs Wiki Getting started#Basic Filesystem Commands**
- **Btrfs Wiki Trees**

Creating a subvolume

To create a subvolume:

```
# btrfs subvolume create /path/to/subvolume
```

Listing subvolumes

To see a list of current subvolumes and their ids under `path`:

```
# btrfs subvolume list -p path
```

Deleting a subvolume

To delete a subvolume:

```
# btrfs subvolume delete /path/to/subvolume
```

Since Linux 4.18, one can also delete a subvolume like a regular directory (`rm -r`, `rmdir`).

Mounting subvolumes

Subvolumes can be mounted like file system partitions using the `subvol=/path/to/subvolume` or `subvolid=objectid` mount flags. For example, you could have a subvolume named `subvol_root` and mount it as `/`. One can mimic traditional file system partitions by creating various subvolumes under the top level of the file system and then mounting them at the appropriate mount points. Thus one can easily restore a file system (or part of it) to a previous state using **#Snapshots**.

Tip: Changing subvolume layouts is made simpler by not using the toplevel subvolume (ID=5) as `/` (which is done by default). Instead, consider creating a subvolume to house your actual data and mounting it as `/`.

Note: From **btrfs(5) § MOUNT OPTIONS**: "Most mount options apply to the **whole filesystem**, and only the options for the first subvolume to be mounted will take effect. This is due to lack of implementation and may change in the future.". See the **Btrfs Wiki FAQ** for which mount options can be used per subvolume.

See **Snapper#Suggested filesystem layout**, **Btrfs SysadminGuide#Managing Snapshots**, and **Btrfs SysadminGuide#Layout** for example file system layouts using subvolumes.

See **btrfs(5)** for a full list of btrfs-specific mount options.

Mounting subvolume as root

To use a subvolume as the root mountpoint specify the subvolume via a **kernel parameter** using `rootflags=subvol=/path/to/subvolume`. Edit the root mountpoint in `/etc/fstab` and specify the mount option `subvol=`. Alternatively the subvolume can be specified with its id, `rootflags=subvolid=objectid` as kernel parameter and `subvolid=objectid` as mount option in `/etc/fstab`.

Changing the default sub-volume

The default sub-volume is mounted if no `subvol=` mount option is provided. To change the default subvolume, do:

```
# btrfs subvolume set-default subvolume-id /
```

where *subvolume-id* can be found by **listing**.

Note: After changing the default subvolume on a system with **GRUB**, you should run `grub-install` again to notify the bootloader of the changes. See **this forum thread**.

Changing the default subvolume with `btrfs subvolume set-default` will make the top level of the filesystem inaccessible, except by use of the `subvol=/` or `subvolid=5` mount options **[6]**.

Quota

Warning: Qgroup is not stable yet and combining quota with (too many) snapshots of subvolumes can cause performance problems, for example when deleting snapshots. Plus there are several more **known issues**.

Quota support in Btrfs is implemented at a subvolume level by the use of quota groups or qgroup: Each subvolume is assigned a quota groups in the form of *0/subvolume_id* by default. However it is possible to create a quota group using any number if desired.

To use qgroups you need to enable quota first using

```
# btrfs quota enable path
```

From this point onwards newly created subvolumes will be controlled by those groups. In order to retrospectively enable them for already existing subvolumes, enable quota normally, then create a qgroup (quota group) for each of those subvolume using their *subvolume_id* and rescan them:

```
# btrfs subvolume list path | cut -d' ' -f2 | xargs -I{} -n1 btrfs qgroup create 0/{} p
ath
# btrfs quota rescan path
```

Quota groups in Btrfs form a tree hierarchy, whereby qgroups are attached to subvolumes. The size limits are set per qgroup and apply when any limit is reached in tree that contains a given subvolume.

Limits on quota groups can be applied either to the total data usage, un-shared data usage, compressed data usage or both. File copy and file deletion may both affect limits since the unshared limit of another qgroup can change if the original volume's files are deleted and only one copy is remaining. For example a fresh snapshot shares almost all the blocks with the original subvolume, new writes to either subvolume will raise towards the exclusive limit, deletions of common data in one volume raises towards the exclusive limit in the other one.

To apply a limit to a qgroup, use the command `btrfs qgroup limit`. Depending on your usage either use a total limit, unshared limit (`-e`) or compressed limit (`-c`). To show usage and limits for a given path within a filesystem use

```
# btrfs qgroup show -reF path
```

Commit interval

The resolution at which data are written to the filesystem is dictated by Btrfs itself and by system-wide settings. Btrfs defaults to a 30 seconds checkpoint interval in which new data are committed to the filesystem. This can be changed by

appending the `commit` mount option in `/etc/fstab` for the btrfs partition.

```
LABEL=arch64 / btrfs defaults,noatime,compress=lzo,commit=120 0 0
```

System-wide settings also affect commit intervals. They include the files under `/proc/sys/vm/*` and are out-of-scope of this wiki article. The kernel documentation for them resides in `Documentation/sysctl/vm.txt`.

SSD TRIM

A Btrfs filesystem is able to free unused blocks from an SSD drive supporting the TRIM command. Starting with kernel version 5.6 there is asynchronous discard support, enabled with mount option `discard=async`. Freed extents are not discarded immediately, but grouped together and trimmed later by a separate worker thread, improving commit latency.

More information about enabling and using TRIM can be found in **Solid State Drives#TRIM**.

Usage

Swap file

Swap files in Btrfs are supported since Linux kernel 5.0.**[7]** The proper way to initialize a swap file is to first create a non-compressed, non-snapshotted subvolume to host the file, *cd* into its directory, then create a zero length file, set the `No_COW` attribute on it with **chattr**, and make sure compression is disabled:

```
# cd /path/to/swapfile
# truncate -s 0 ./swapfile
# chattr +C ./swapfile
# btrfs property set ./swapfile compression none
```

See **Swap file#Swap file creation** for further configuration. Configuring hibernation to a swap file is described in **Power management/Suspend and hibernate#Hibernation into swap file on Btrfs**.

Note: Since Linux kernel 5.0, Btrfs has native swap file support with some limitations:

- The swap file cannot be on a snapshotted subvolume. The proper procedure is to create a new subvolume to place the swap file in.
- It does not support swap files on file systems that span multiple devices. See **Btrfs wiki: Does btrfs support swap files?** and **Arch forums discussion**.

Displaying used/free space

General linux userspace tools such as `df` will inaccurately report free space on a Btrfs partition. It is recommended to use `btrfs filesystem usage` to query Btrfs partitions. For example:

```
# btrfs filesystem usage /
```

Note: The `btrfs filesystem usage` command does not currently work correctly with `RAID5/RAID6` RAID levels.

See **[8]** for more information.

Defragmentation

Btrfs supports online defragmentation through the mount option `autodefrag`, see **btrfs(5) § MOUNT OPTIONS**. To manually defragment your root, use:

```
# btrfs filesystem defragment -r /
```

Using the above command without the `-r` switch will result in only the metadata held by the subvolume containing the directory being defragmented. This allows for single file defragmentation by simply specifying the path.

Defragmenting a file which has a COW copy (either a snapshot copy or one made with `cp --reflink` or bcp) plus using the `-c` switch with a compression algorithm may result in two unrelated files effectively increasing the disk usage.

RAID

Btrfs offers native "RAID" for **#Multi-device file systems**. Notable features which set btrfs RAID apart from **mdadm** are self-healing redundant arrays and online balancing. See **the Btrfs wiki page** for more information. The Btrfs sysadmin page also **has a section** with some more technical background.

Warning: Parity RAID (RAID 5/6) code has multiple serious data-loss bugs in it. See the Btrfs Wiki's **RAID5/6 page** and a bug report on **linux-btrfs mailing list** for more detailed information. In June 2020, somebody posted a **comprehensive list of current issues** and a **helpful recovery guide**.

Scrub

This article or section needs expansion.

Reason: **Does Btrfs scrub examine subvolume or the device that subvolume resides on?**
(Discuss in **Talk:Btrfs#**)

The **Btrfs Wiki Glossary** says that Btrfs scrub is "[a]n online filesystem checking tool. Reads all the data and metadata on the filesystem, and uses checksums and the duplicate copies from RAID storage to identify and repair any corrupt data."

Note: A running scrub process will prevent the system from suspending, see **this thread** [**dead link** 2020-02-23] for details.

Start manually

To start a (background) scrub on the filesystem which contains `/`:

```
# btrfs scrub start /
```

To check the status of a running scrub:

```
# btrfs scrub status /
```

Start with a service or timer

The **btrfs-progs** package brings the `btrfs-scrub@.timer` unit for monthly scrubbing the specified mountpoint. **Enable** the timer with an escaped path, e.g. `btrfs-scrub@-.timer` for `/` and `btrfs-scrub@home.timer` for `/home`. You can use `systemd-escape -p /path/to/mountpoint` to escape the path, see **systemd-escape(1)** for details.

You can also run the scrub by **starting** `btrfs-scrub@.service` (with the same encoded path). The advantage of this over `btrfs scrub` (as the root user) is that the results of the scrub will be logged in the **systemd journal**.

Balance

"A balance passes all data in the filesystem through the allocator again. It is primarily intended to rebalance the data in the filesystem across the devices when a device is added or removed. A balance will regenerate missing copies for the redundant RAID levels, if a device has failed." **[9]** See **Upstream FAQ page**.

On a single-device filesystem a balance may be also useful for (temporarily) reducing the amount of allocated but unused (meta)data chunks. Sometimes this is needed for fixing **"filesystem full" issues**.

```
# btrfs balance start /
# btrfs balance status /
```

Snapshots

"A snapshot is simply a subvolume that shares its data (and metadata) with some other subvolume, using btrfs's COW capabilities." See **Btrfs Wiki SysadminGuide#Snapshots** for details.

To create a snapshot:

```
# btrfs subvolume snapshot source [dest/]name
```

To create a readonly snapshot add the `-r` flag. To create writable version of a readonly snapshot, simply create a snapshot of it.

Note: Snapshots are not recursive. Every nested subvolume will be an empty directory inside the snapshot.

Send/receive

A subvolume can be sent to stdout or a file using the `send` command. This is usually most useful when piped to a Btrfs `receive` command. For example, to send a snapshot named `/root_backup` (perhaps of a snapshot you made of `/` earlier) to `/backup` you would do the following:

```
# btrfs send /root_backup | btrfs receive /backup
```

The snapshot that is sent *must* be readonly. The above command is useful for copying a subvolume to an external device (e.g. a USB disk mounted at `/backup` above).

You can also send only the difference between two snapshots. For example, if you have already sent a copy of `root_backup` above and have made a new readonly snapshot on your system named `root_backup_new`, then to send only the incremental difference to `/backup` do:

```
# btrfs send -p /root_backup /root_backup_new | btrfs receive /backup
```

Now a new subvolume named `root_backup_new` will be present in `/backup`.

See **Btrfs Wiki's Incremental Backup page** on how to use this for incremental backups and for tools that automate the process.

Deduplication

Using copy-on-write, Btrfs is able to copy files or whole subvolumes without actually copying the data. However whenever a file is altered a new *proper* copy is created. Deduplication takes this a step further, by actively identifying blocks of data which share common sequences and combining them into an extent with the same copy-on-write semantics.

Tools dedicated to deduplicate a Btrfs formatted partition include **duperemove**, **bedup**[AUR] and *btrfs-dedup*. One may also want to merely deduplicate data on a file based level instead using e.g. **rmlint**, **jdupes**[AUR] or **dduper-git**[AUR]. For an overview of available features of those programs and additional information have a look at the **upstream Wiki entry**.

Furthermore Btrfs developers are working on inband (also known as synchronous or inline) deduplication, meaning deduplication done when writing new data to the filesystem. Currently it is still an experiment which is developed out-of-tree. Users willing to test the new feature should read the **appropriate kernel wiki page**.

Known issues

A few limitations should be known before trying.

Encryption

Btrfs has no built-in encryption support, but this **may** come in the future. Users can encrypt the partition before running `mkfs.btrfs`. See **dm-crypt/Encrypting an entire system#Btrfs subvolumes with swap**.

Existing Btrfs file systems can use something like **EncFS** or **TrueCrypt**, though perhaps without some of Btrfs' features.

TLP

Using TLP requires special precautions in order to avoid filesystem corruption. Refer to the **according TLP section** for more information.

btrfs check issues

The tool `btrfs check` has known issues and should not be run without further reading, see section **#btrfs check**.

Tips and tricks

Partitionless Btrfs disk

Warning:

- Most users do not want this type of setup and instead should install Btrfs on a regular partition. Furthermore, GRUB strongly discourages installation to a partitionless disk.
- Since **grub** 2.04, GRUB's `core.img` is too big to fit in Btrfs VBR. See **FS#63656**.

Btrfs can occupy an entire data storage device, replacing the **MBR** or **GPT** partitioning schemes, using **subvolumes** to simulate partitions. However, using a partitionless setup is not required to simply **create a Btrfs filesystem** on an existing **partition** that was created using another method. There are some limitations to partitionless single disk setups:

- Cannot place other **file systems** on another partition on the same disk.
- If using a Linux kernel version before 5.0, you cannot use **swap area** as Btrfs did not support **swap files** pre-5.0 and there is no place to create **swap partition**
- Cannot use **UEFI** to boot.

To overwrite the existing partition table with Btrfs, run the following command:

```
# mkfs.btrfs /dev/sdX
```

For example, use `/dev/sda` rather than `/dev/sda1`. The latter would format an existing partition instead of replacing the entire partitioning scheme. Because the root partition is Btrfs, make sure `btrfs` is compiled into the kernel, or put `btrfs` into **mkinitcpio.conf#MODULES** and **regenerate the initramfs**.

Install the **boot loader** like you would for a data storage device with a **Master Boot Record**. See **Syslinux#Manual install** or **GRUB/Tips and tricks#Install to partition or partitionless disk**. If your kernel does not boot due to `Failed to mount /sysroot.`, please add `GRUB_PRELOAD_MODULES="btrfs"` in `/etc/default/grub` and generate the grub configuration (**GRUB#Generate the main configuration file**).

Ext3/4 to Btrfs conversion

Warning: There are many reports on the btrfs mailing list about incomplete/corrupt/broken conversions. Make sure you have *working* backups of any data you cannot afford to lose. See **Conversion from Ext3** on the btrfs wiki for more information.

Warning: There is a bug in btrfs-progs 5.6.1 and before, that will yield a btrfs filesystem with wrong size for the last block group, thus preventing to mount the newly converted btrfs. This bug is fixed in btrfs-progs 5.7 in **this commit**. Please use btrfs-convert from btrfs-progs 5.7-1 and above.

Boot from an install CD, then convert by doing:

```
# btrfs-convert /dev/partition
```

Mount the partion and test the conversion by checking the files. Be sure to change the `/etc/fstab` to reflect the change (**type** to `btrfs` and **fs_passno** [the last field] to `0` as Btrfs does not do a file system check on boot). Also note that the UUID of the partition will have changed, so update fstab accordingly when using UUIDs. `chroot` into the system and rebuild the GRUB menu list (see **Install from existing Linux** and **GRUB** articles). If converting a root filesystem, while still chrooted run `mkinitcpio -p linux` to regenerate the initramfs or the system will not successfully boot. If you get stuck in grub with 'unknown filesystem' try reinstalling grub with `grub-install /dev/partition` and regenerate the config as well `grub-mkconfig -o /boot/grub/grub.cfg`.

Note: If there is anything wrong, either unable to mount or write files to the newly converted btrfs, there is always the option to rollback as long as the backup subvolume `/ext2_saved` is still there. Use `btrfs-convert -r /dev/partition` command to rollback, this will discard any modifications to the newly converted btrfs filesystem.

After confirming that there are no problems, complete the conversion by deleting the backup `ext2_saved` sub-volume. Note that you cannot revert back to ext3/4 without it.

```
# btrfs subvolume delete /ext2_saved
```

Finally **balance** the file system to reclaim the space.

Remember that some applications which were installed prior have to be adapted to Btrfs. Notably **TLP#Btrfs** needs special care to avoid filesystem corruption but other applications may profit from certain features as well.

Checksum hardware acceleration

The factual accuracy of this article or section is disputed.

Reason: According to [10], the CRC algorithm printed by the following command simply matches "*whatever crypto library is currently loaded at the time*", and "*can change arbitrarily while the file system is loaded*". So this method should not be relied upon in order to determine which CRC algorithm Btrfs is currently using. (Discuss in Talk:Btrfs#)

CRC32 is a new instruction in Intel SSE4.2. To verify if Btrfs checksum is hardware accelerated:

```
$ dmesg | grep crc32c
```

```
Btrfs loaded, crc32c=crc32c-intel
```

If you see `crc32c=crc32c-generic`, it is probably because your root partition is Btrfs, and you will have to compile `crc32c-intel` into the kernel to make it work. Putting `crc32c-intel` into **mkinitcpio.conf** does *not* work.

Corruption recovery

Warning: The tool `btrfs check` has known issues, see section **#btrfs check**

btrfs-check cannot be used on a mounted file system. To be able to use *btrfs-check* without booting from a live USB, add it to the initial ramdisk:

```
/etc/mkinitcpio.conf
```

```
BINARIES=("/usr/bin/btrfs")
```

Regenerate the initramfs.

Then if there is a problem booting, the utility is available for repair.

Note: If the fsck process has to invalidate the space cache (and/or other caches?) then it is normal for a subsequent boot to hang up for a while (it may give console messages about btrfs-transaction being hung). The system should recover from this after a while.

See the **Btrfs Wiki page** for more information.

Booting into snapshots

In order to boot into a snapshot, the same procedure applies as for mounting a subvolume as your root parition, as given in section **mounting a subvolume as your root partition**, because snapshots can be mounted like subvolumes. If using GRUB you can automatically populate your boot menu with btrfs snapshots when regenerating the configuration file with the help of **grub-btrfs** or **grub-btrfs-git**[AUR].

Use Btrfs subvolumes with systemd-nspawn

See the **Systemd-nspawn#Use Btrfs subvolume as container root** and **Systemd-nspawn#Use temporary Btrfs snapshot of container** articles.

Troubleshooting

See the **Btrfs Problem FAQ** for general troubleshooting.

GRUB

Partition offset

The offset problem may happen when you try to embed `core.img` into a partitioned disk. It means that **it is OK** to embed GRUB's `core.img` into a Btrfs pool on a partitionless disk (e.g. `/dev/sdX`) directly.

GRUB can boot Btrfs partitions, however the module may be larger than other **file systems**. And the `core.img` file made by `grub-install` may not fit in the first 63 sectors (31.5KiB) of the drive between the MBR and the first partition. Up-to-date partitioning tools such as `fdisk` and `gdisk` avoid this issue by offsetting the first partition by roughly 1MiB or 2MiB.

Missing root

The factual accuracy of this article or section is disputed.

Reason: Suggests editing a non-configuration file manually. (Discuss in **Talk:Btrfs#Should not suggest to edit files in /usr/share**)

Users experiencing the following: `error no such device: root` when booting from a RAID style setup then edit /usr/share/grub/grub-mkconfig_lib and remove both quotes from the line `echo " search --no-floppy --fs-uuid --set=root ${hints} ${fs_uuid}"`. Regenerate the config for grub and the system should boot without an error.

BTRFS: open_ctree failed

As of November 2014 there seems to be a bug in **systemd** or **mkinitcpio** causing the following error on systems with multi-device Btrfs filesystem using the `btrfs` hook in `mkinitcpio.conf`:

```
BTRFS: open_ctree failed
mount: wrong fs type, bad option, bad superblock on /dev/sdb2, missing codepage or help
er program, or other error

In some cases useful info is found in syslog - try dmesg|tail or so.

You are now being dropped into an emergency shell.
```

A workaround is to remove `btrfs` from the `HOOKS` array in `/etc/mkinitcpio.conf` and instead add `btrfs` to the `MODULES` array. Then **regenerate the initramfs** and reboot.

You will get the same error if you try to mount a raid array without one of the devices. In that case you must add the `degraded` mount option to `/etc/fstab`. If your root resides on the array, you must also add `rootflags=degraded` to your **kernel parameters**.

As of August 2016, a potential workaround for this bug is to mount the array by a single drive only in `/etc/fstab`, and allow btrfs to discover and append the other drives automatically. Group-based identifiers such as UUID and LABEL appear to contribute to the failure. For example, a two-device RAID1 array consisting of 'disk1' and disk2' will have a UUID allocated to it, but instead of using the UUID, use only `/dev/mapper/disk1` in `/etc/fstab`. For a more detailed explanation, see the following **blog post** [dead link 2020-05-24 ⓘ].

Another possible workaround is to remove the `udev` hook in **mkinitcpio.conf** and replace it with the `systemd` hook. In this case, `btrfs` should *not* be in the `HOOKS` or `MODULES` arrays.

See the **original forums thread** and **FS#42884** for further information and discussion.

btrfs check

Warning: Since Btrfs is under heavy development, especially the `btrfs check` command, it is highly recommended to create a **backup** and consult the **Btrfsck documentation** before executing `btrfs check` with the `--repair` switch.

The ***btrfs check*** command can be used to check or repair an unmounted Btrfs filesystem. However, this repair tool is still immature and not able to repair certain filesystem errors even those that do not render the filesystem unmountable.

See also

- **Official site**

  Btrfs Wiki

Displaying used/free space in btrfs:

List how much space has been allocated on your filesystem:

```
            $ sudo btrfs filesystem show /
^same as    $ sudo btrfs fi show /
```

List Complete filesystem usage by data and metadata:

```
           $  sudo btrfs filesystem usage /
^ same as $  sudo btrfs fi usage /
```

Balance btrfs:

```
    $  btrfs balance start /<mount point of  device>
or $  btrfs bal start /<mount point of device>
```

See progress of btrfs balance:

```
                $  btrfs balance status /<mount point of device>
             or  $  watch btrfs balance status /<mount point of device>
```

Defragment Btrfs Filesystem:

```
btrfs filesystem defragment -r /<mount point of device>
```

Using Btrfs with Multiple Devices

Multiple devices

A Btrfs filesystem can be created on top of many devices, and more devices can be added after the FS has been created.

By default, metadata will be mirrored across two devices and data will be striped across all of the devices present. This is equivalent to `mkfs.btrfs -m raid1 -d raid0`.

If only one device is present, metadata will be duplicated on that one device. For HDD `mkfs.btrfs -m dup -d single`, for SSD (or non-rotational device) `mkfs.btrfs -m single -d single`.

Current status

Btrfs can add and remove devices online, and freely convert between RAID levels after the FS has been created.

Btrfs supports RAID 0, RAID 1, RAID 10, RAID 5 and RAID 6 (but see the section below about RAID 5/6), and it can also duplicate metadata or data on a single spindle or multiple disks. When blocks are read in, checksums are verified. If there are any errors, Btrfs tries to read from an alternate copy and will repair the broken copy if the alternative copy succeeds.

See the Gotchas page for some current issues when using btrfs with multiple volumes of differing sizes in a RAID 1 style setup.

The RAID 1 profile also allows for 2, 3, or 4 copies of redundant data copies, called RAID 1, RAID 1C3, and RAID 1C4 respectively. See Manpage/mkfs.btrfs for more details on this and other RAID profiles.

RAID 5 and RAID 6

Please read the parity RAID status page first: RAID56.

Note that the minimum number of devices required for RAID 5 is 3. For a RAID 6, the minimum is 4 devices.

Filesystem creation

`mkfs.btrfs` will accept more than one device on the command line. It has options to control the RAID configuration for data (`-d`) and metadata (`-m`). Valid choices are `raid0`, `raid1`, `raid10`, `raid5`, `raid6`, `single` and `dup`. The option `-m single` means that no duplication is done, which may be desired when using hardware RAID.

RAID 10 requires at least 4 devices.

```
# Create a filesystem across four drives (metadata mirrored, linear data allocation)
mkfs.btrfs -d single /dev/sdb /dev/sdc /dev/sdd /dev/sde

# Stripe the data without mirroring, metadata are mirrored
mkfs.btrfs -d raid0 /dev/sdb /dev/sdc

# Use raid10 for both data and metadata
mkfs.btrfs -m raid10 -d raid10 /dev/sdb /dev/sdc /dev/sdd /dev/sde

# Don't duplicate metadata on a single drive (default on single SSDs)
mkfs.btrfs -m single /dev/sdb
```

If you want to use devices of different sizes, striped RAID levels (RAID 0, RAID 10, RAID 5, RAID 6) may not use all of the available space on the devices. (See btrfs-space-calculator or btrfs disk usage calculator) Non-striped equivalents may give you a more effective use of space (single instead of RAID 0, RAID 1 instead of RAID 10).

```
# Use full capacity of multiple drives with different sizes (metadata mirrored, data not mirrored and not striped)
mkfs.btrfs -d single /dev/sdb /dev/sdc
```

Once you create a multi-device filesystem, you can use any device in the FS for the mount command:

```
mkfs.btrfs /dev/sdb /dev/sdc /dev/sde
mount /dev/sde /mnt
```

If you want to mount a multi-device filesystem using a loopback device, it's not sufficient to use `mount -o loop`. Instead, you'll have to set up the loopbacks manually:

```
# Create and mount a filesystem made of several disk images
mkfs.btrfs img0 img1 img2
losetup /dev/loop0 img0
losetup /dev/loop1 img1
losetup /dev/loop2 img2
mount /dev/loop0 /mnt/btrfs
```

After a reboot or reloading the btrfs module, you'll need to use `btrfs device scan` to discover all multi-device filesystems on the machine (see below)

The UseCases page gives a few quick recipes for filesystem creation.

Device scanning

`btrfs device scan` is used to scan all of the block devices under `/dev` and probe for Btrfs volumes. This is required after loading the btrfs module if you're running with more than one device in a filesystem.

```
# Scan all devices
btrfs device scan

# Scan a single device
btrfs device scan /dev/sdb
```

`btrfs filesystem show` will print information about all of the btrfs filesystems on the machine.

Adding new devices

`btrfs filesystem show` gives you a list of all the btrfs filesystems on the systems and which devices they include.

`btrfs device add` is used to add new devices to a mounted filesystem.

`btrfs balance` can balance (restripe) the allocated extents across all of the existing devices. For example, with an existing filesystem mounted at `/mnt`, you can add the device `/dev/sdc` to it with:

```
btrfs device add /dev/sdc /mnt
```

At this point we have added our device to the filesystem, but all of the metadata and data are still stored on the original device(s). The filesystem can be balanced to spread the extents across all the devices.

```
btrfs balance start /mnt
```

The `btrfs balance` operation will take some time. It reads in all of the FS data and metadata and rewrites it across all the available devices. Depending on the usage scenario that is not needed, one example would be adding a new device to a RAID 1 filesystem whose existing devices still have enough space left. However, in other cases a balancing might be needed. Take a closer look at available filter options that can help reducing the needed time and scope of the balance.

Conversion

A non-RAID filesystem is converted to RAID by adding a device and running a balance filter that will change the chunk allocation profile.

For example, to convert an existing single device system (`/dev/sdb1`) into a 2 device RAID 1 (to protect against a single disk failure):

```
mount /dev/sdb1 /mnt
btrfs device add /dev/sdc1 /mnt
btrfs balance start -dconvert=raid1 -mconvert=raid1 /mnt
```

If the metadata is not converted from the single-device default, it remains as DUP, which does not guarantee that copies of block are on separate devices. If data is not converted it does not have any redundant copies at all.

Removing devices

`btrfs device delete` is used to remove devices online. It redistributes any extents in use on the device being removed to the other devices in the filesystem. Example:

```
mkfs.btrfs /dev/sdb /dev/sdc /dev/sdd /dev/sde
mount /dev/sdb /mnt
# Put some data on the filesystem here
btrfs device delete /dev/sdc /mnt
```

Replacing failed devices

Using `btrfs replace`

When you have a device that's in the process of failing or has failed in a RAID array you should use the `btrfs replace` command rather than adding a new device and removing the failed one. This is a newer technique that worked for me when adding and deleting devices didn't however it may be helpful to consult the mailing list of irc channel before attempting recovery.

First list the devices in the filesystem, in this example we have one missing device (5 devices out of 6 are shown). The devids in the example are not sequential since `add` and `remove` have been used previously instead of `replace`.

```
user@host:~$ sudo btrfs filesystem show
Label: none  uuid: 67b4821f-16e0-436d-b521-e4ab2c7d3ab7
    Total devices 6 FS bytes used 5.47TiB
    devid    1 size 1.81TiB used 1.71TiB path /dev/sda3
    devid    3 size 1.81TiB used 1.71TiB path /dev/sdb3
    devid    4 size 1.82TiB used 1.72TiB path /dev/sdc1
    devid    6 size 1.82TiB used 1.72TiB path /dev/sdd1
    devid    8 size 2.73TiB used 2.62TiB path /dev/sde1
    *** Some devices missing
```

Before replacing the device you will need to mount the array, if you have a missing device then you will need to use the following command:

```
  sudo mount -o degraded /dev/sda3 /mnt
```

Now replace the absent device with the new drive `/dev/sdf1` on the filesystem currently mounted on `/mnt` (since the device is absent, you can use any devid number that isn't present; 2,5,7,9 would all work the same):

```
  sudo btrfs replace start 7 /dev/sdf1 /mnt
```

This will start the replacement process in the background, you can monitor the status using the `btrfs replace status` command. The `status` command updates the status continuously, you can ctrl+c to exit it at any time. At first progress will be shown, on completing you'll see the following output:

```
  user@host:~$ sudo btrfs replace status /mnt
  Started on 27.Mar 22:34:20, finished on 28.Mar 06:36:15, 0 write errs, 0 uncorr. read errs
```

Caveats

- If replacing an existing device with a smaller device, you should use `btrfs fi resize` first. If replacing a device with a larger device, you should use `resize` after.
- The error counters appear to be inaccurate, I had errors in my recovery reported in dmesg but not in the replace status output. The following command will parse out all the names of damaged files currently in the dmesg log, it only works on the messages currently in the kernel log ring buffer so if you have received too many log messages it will not be complete:

```
  dmesg | grep BTRFS | grep path | sed -e 's/^.*path: //;s/)$//' | sort | uniq
```

Using add and delete

The example above can be used to remove a failed device if the super block can still be read. But, if a device is missing or the super block has been corrupted, the filesystem will need to be mounted in degraded mode:

```
mkfs.btrfs -m raid1 /dev/sdb /dev/sdc /dev/sdd /dev/sde
#
# sdd is destroyed or removed, use -o degraded to force the mount
# to ignore missing devices
#
mount -o degraded /dev/sdb /mnt
#
# 'missing' is a special device name
#
btrfs device delete missing /mnt
```

`btrfs device delete missing` tells btrfs to remove the first device that is described by the filesystem metadata but not present when the FS was mounted.

In case of *RAID XX* layout, you cannot go below the minimum number of the device required. So before removing a device (even the *missing* one) you may need to add a new one. For example if you have a RAID 1 layout with **two** devices, and a device fails, you must:

- mount in degraded mode
- add a new device
- remove the *missing* device

Registration in `/etc/fstab`

If you don't have an initrd, or your initrd doesn't perform a btrfs device scan, you can still mount a multi-volume btrfs filesystem by passing *all* the devices in the filesystem explicitly to the mount command. A suitable `/etc/fstab` entry would be:

```
/dev/sdb      /mnt     btrfs     device=/dev/sdb,device=/dev/sdc,device=/dev/sdd,device=/dev/sde     0 0
```

Using `device` is **not** recommended, as it is sensitive to device names changing that is affected by the order of discovery of the devices. You should *really* be using a initramfs. Most modern distributions will do this for you automatically if you install their own btrfs-progs package.

Using `device=PARTUUID=...` with GPT partition UUIDs would also work and be less fragile. All these options can also be set from the kernel command line, through root=/fstype=/rootflags=.

Btrfs

How to Set Up Btrfs RAID

3 months ago • by Shahriar Shovon

Btrfs is a modern Copy-on-Write (CoW) filesystem with built-in RAID support. So, you do not need any third-party tools to create software RAIDs on a Btrfs filesystem.

The Btrfs filesystem keeps the filesystem metadata and data separately. You can use different RAID levels for the data and metadata at the same time. This is a major advantage of the Btrfs filesystem.

This article shows you how to set up Btrfs RAIDs in the RAID-0, RAID-1, RAID-1C3, RAID-1C4, RAID-10, RAID-5, and RAID-6 configurations.

Abbreviations

- **Btrfs** – B-tree Filesystem
- **RAID** – Redundant Array of Inexpensive Disks/Redundant Array of Independent Disks
- **GB** – Gigabyte
- **TB** – Terabyte
- **HDD** – Hard Disk Drive
- **SSD** – Solid-State Drive

Prerequisites

To try out the examples included in this article:

- You must have the Btrfs filesystem installed on your computer.
- You will need at least four same-capacity HDDs/SSDs to try out the different RAID configurations.

In my Ubuntu machine, I have added four HDDs (**sdb, sdc, sdd, sde**). Each of them is 20 GB in size.

```
$ sudo lsblk
```

Note: Your HDDs/SSDs may have different names than mine. So, be sure to replace them with yours from now on.

For assistance with installing the Btrfs filesystem in Ubuntu, check out the article Install and Use Btrfs on Ubuntu 20.04 LTS.

For assistance with installing the Btrfs filesystem in Fedora, check out the article Install and Use Btrfs on Fedora 33.

Btrfs Profiles

A Btrfs profile is used to tell the Btrfs filesystem how many copies of the data/metadata to keep and what RAID levels to use for the data/metadata. The Btrfs filesystem contains many profiles. Understanding them will help you to configure a Btrfs RAID just the way you want.

The available Btrfs profiles are as follows:

single: If the **single** profile is used for the data/metadata, only one copy of the data/metadata will be stored in the filesystem, even if you add multiple storage devices to the filesystem. So, **100%** of the disk space of each of the storage devices added to the filesystem can be utilized.

dup: If the **dup** profile is used for the data/metadata, each of the storage devices added to the filesystem will keep two copies of the data/metadata. So, **50%** of the disk space of each of the storage devices added to the filesystem can be utilized.

raid0: In the **raid0** profile, the data/metadata will be split evenly across all the storage devices added to the filesystem. In this setup, there will be no redundant (duplicate) data/metadata. So, **100%** of the disk space of each of the storage devices added to the filesystem can be used. If in any case one of the storage devices fails, the entire filesystem will be corrupted. You will need at least two storage devices to set up the Btrfs filesystem in the **raid0** profile.

raid1: In the **raid1** profile, two copies of the data/metadata will be stored in the storage devices added to the filesystem. In this setup, the RAID array can survive one drive failure. But, you can use only **50%** of the total disk space. You will need at least two storage devices to set up the Btrfs filesystem in the **raid1** profile.

raid1c3: In the **raid1c3** profile, three copies of the data/metadata will be stored in the storage devices added to the filesystem. In this setup, the RAID array can survive two drive failures, but you can use only **33%** of the total disk space. You will need at least three storage devices to set up the Btrfs filesystem in the **raid1c3** profile.

raid1c4: In the **raid1c4** profile, four copies of the data/metadata will be stored in the storage devices added to the filesystem. In this setup, the RAID array can survive three drive failures, but you can use only **25%** of the total disk space. You will need at least four storage devices to set up the Btrfs filesystem in the **raid1c4** profile.

raid10: In the **raid10** profile, two copies of the data/metadata will be stored in the storage devices added to the filesystem, as in the **raid1** profile. Also, the data/metadata will be split across the storage devices, as in the **raid0** profile.

The **raid10** profile is a hybrid of the **raid1** and **raid0** profiles. Some of the storage devices form **raid1** arrays and some of these **raid1** arrays are used to form a **raid0** array. In a **raid10** setup, the filesystem can survive a single drive failure in each of the **raid1** arrays.

You can use **50%** of the total disk space in the **raid10** configuration. You will need at least four storage devices to set up the Btrfs filesystem in the **raid10** profile.

raid5: In the **raid5** profile, one copy of the data/metadata will be split across the storage devices. A single parity will be calculated and distributed among the storage devices of the RAID array.

In a **raid5** configuration, the filesystem can survive a single drive failure. If a drive fails, you can add a new drive to the filesystem and the lost data will be calculated from the distributed parity of the running drives.

You can use 1**00x(N-1)/N %** of the total disk spaces in the **raid5** configuration. Here, **N** is the number of storage devices added to the filesystem. You will need at least three storage devices to set up the Btrfs filesystem in the **raid5** profile.

raid6: In the **raid6** profile, one copy of the data/metadata will be split across the storage devices. Two parities will be calculated and distributed among the storage devices of the RAID array.

In a **raid6** configuration, the filesystem can survive two drive failures at once. If a drive fails, you can add a new drive to the filesystem, and the lost data will be calculated from the two distributed parities of the running drives.

You can use **100x(N-2)/N %** of the total disk space in the **raid6** configuration. Here, **N** is the number of storage devices added to the filesystem. You will need at least four storage devices to set up the Btrfs filesystem in the **raid6** profile.

Creating a Mount Point

You need to create a directory to mount the Btrfs filesystem that you will create in the next sections of this article.

To create the directory/mount point /data, run the following command:

```
$ sudo mkdir -v /data
```

Setting Up RAID-0

In this section, you will learn how to set up a Btrfs RAID in the RAID-0 configuration using four HDDs (**sdb, sdc, sdd, and sde**). The HDDs are 20 GB in size.

```
$ sudo lsblk
```

To create a Btrfs RAID in the RAID-0 configuration using four HDDs (**sdb, sdc, sdd,** and **sde**) run the following command:

```
$ sudo mkfs.btrfs -L data -d raid0 -m raid0 -f /dev/sdb /dev/sdc /dev/sdd /dev/sde
```

Here,

- The –**L** option is used to set the filesystem label **data**.
- The –**d** option is used to set the RAID profile **raid0** for the filesystem data.
- The –**m** option is used to set the RAID profile **raid0** for the filesystem metadata.
- The –**f** option is used to force the creation of the Btrfs filesystem, even if any of the HDDs have an existing filesystem.

The Btrfs filesystem **data** in the RAID-0 configuration should now be created, as you can see in the screenshot below.

You can mount the Btrfs RAID using any HDD/SSD you used to create the RAID.

For example, I used the HDDs **sdb, sdc, sdd,** and **sde** to create the Btrfs RAID in the RAID-0 configuration.

So, I can mount the Btrfs filesystem **data** in the **/data** directory using the HDD **sdb**, as follows:

```
$ sudo mount /dev/sdb /data
```

As you can see, the Btrfs RAID is mounted in the **/data** directory.

```
$ sudo df -h /data
```

To find the filesystem usage information of the **data** Btrfs filesystem mounted in the **/data** directory, run the following command:

```
$ sudo btrfs filesystem usage /data
```

As you can see,

The RAID size (**Device size**) is **80 GB** (4×20 GB per HDD).

About **78.98 GB** (**Free (estimated)**) of **80 GB** of disk space can be used in the RAID-0 configuration.

Only one copy of the data (**Data ratio**) and one copy of the metadata (**Metadata ratio**) will be stored in the Btrfs filesystem in the RAID-0 configuration.

As the Btrfs RAID is working, you can unmount it from the **/data** directory, as follows:

```
$ sudo umount /data
```

Setting Up RAID-1

In this section, you will learn how to set up a Btrfs RAID in the RAID-1 configuration using four HDDs (**sdb, sdc, sdd**, and **sde**). The HDDs are 20 GB in size.

```
$ sudo lsblk
```

To create a Btrfs RAID in the RAID-1 configuration using four HDDs (**sdb, sdc, sdd,** and **sde**), run the following command:

```
$ sudo mkfs.btrfs -L data -d raid1 -m raid1 -f /dev/sdb /dev/sdc /dev/sdd /dev/sde
```

Here,

- The –**L** option is used to set the filesystem label **data**.
- The –**d** option is used to set the RAID profile **raid1** for the filesystem data.
- The –**m** option is used to set the RAID profile **raid1** for the filesystem metadata.
- The –**f** option is used to force the creation of the Btrfs filesystem, even if any of the HDDs have an existing filesystem.

The Btrfs filesystem data in the RAID-1 configuration should now be created, as you can see in the screenshot below.

You can mount the Btrfs RAID using any HDD/SSD you used to create the RAID.

For example, I used the HDDs **sdb, sdc, sdd,** and **sde** to create the Btrfs RAID in the RAID-1

configuration.

I can mount the Btrfs filesystem **data** in the **/data** directory using the HDD **sdb**, as follows:

```
$ sudo mount /dev/sdb /data
```

As you can see, the Btrfs RAID is mounted in the /data directory.

```
$ sudo df -h /data
```

To find the filesystem usage information of the data Btrfs filesystem mounted in the **/data** directory, run the following command:

```
$ sudo btrfs filesystem usage /data
```

As you can see,

The RAID size (**Device size**) is **80 GB** (4×20 GB per HDD).

About **38.99 GB** (**Free (estimated)**) of **80 GB** of disk space can be used in the RAID-1 configuration.

In the RAID-1 configuration, two copies of the data (**Data ratio**) and two copies of the metadata (**Metadata ratio**) will be stored in the Btrfs filesystem.

As the Btrfs RAID is working, you can unmount it from the **/data** directory, as follows:

```
$ sudo umount /data
```

Setting Up RAID-1C3

In this section, you will learn how to set up a Btrfs RAID in the RAID-1C3 configuration using four HDDs (sdb, sdc, sdd, and sde). The HDDs are 20 GB in size

```
$ sudo lsblk
```

To create a Btrfs RAID in the RAID-1C3 configuration using the four HDDs **sdb, sdc, sdd,** and **sde**, run the following command:

```
$ sudo mkfs.btrfs -L data -d raid1c3 -m raid1c3 -f /dev/sdb /dev/sdc /dev/sdd /dev/sde
```

Here,

- The –**L** option is used to set the filesystem label data.
- The –**d** option is used to set the RAID profile **raid1c3** for the filesystem **data**.
- The –**m** option is used to set the RAID profile **raid1c3** for the filesystem metadata.
- The –**f** option is used to force the creation of the Btrfs filesystem, even if any of the HDDs have an existing filesystem.

The Btrfs filesystem **data** in the RAID-1C3 configuration should now be created, as you can see in the screenshot below.

You can mount the Btrfs RAID using any HDD/SSD you used to create the RAID.

For example, I used the HDDs **sdb, sdc, sdd,** and **sde** to create the Btrfs RAID in the RAID-1C3 configuration.

So, I can mount the Btrfs filesystem **data** in the **/data** directory using the HDD **sdb**, as follows:

```
$ sudo mount /dev/sdb /data
```

As you can see, the Btrfs RAID is mounted in the **/data** directory.

```
$ sudo df -h /data
```

To find the filesystem usage information of the **data** Btrfs filesystem mounted in the **/data** directory, run the following command:

```
$ sudo btrfs filesystem usage /data
```

As you can see,

The RAID size (**Device size**) is **80 GB** (4×20 GB per HDD).

About **25.66 GB** (**Free (estimated)**) of **80 GB** of disk space can be used in the RAID-1C3 configuration.

In the RAID-1C3 configuration, three copies of the data (**Data ratio**) and three copies of the metadata (**Metadata ratio**) will be stored in the Btrfs filesystem.

As the Btrfs RAID is working, you can unmount it from the **/data** directory, as follows:

```
$ sudo umount /data
```

Setting Up RAID-1C4

In this section, you will learn how to set up a Btrfs RAID in the RAID-1C4 configuration using the four HDDs **sdb, sdc, sdd,** and **sde**. The HDDs are 20 GB in size.

```
$ sudo lsblk
```

To create a Btrfs RAID in the RAID-1C4 configuration using the four HDDs **sdb, sdc, sdd,** and **sde**, run the following command:

```
$ sudo mkfs.btrfs -L data -d raid1c4 -m raid1c4 -f /dev/sdb /dev/sdc /dev/sdd /dev/sde
```

Here,

- The –**L** option is used to set the filesystem label **data**.
- The –**d** option is used to set the RAID profile **raid1c4** for the filesystem data.
- The –**m** option is used to set the RAID profile **raid1c4** for the filesystem metadata.
- The –**f** option is used to force the creation of the Btrfs filesystem, even if any of the HDDs have an existing filesystem.

The Btrfs filesystem **data** in the RAID-1C4 configuration should now be created, as you can see in the screenshot below.

You can mount the Btrfs RAID using any HDD/SSD you used to create the RAID.

For example, I used the HDDs **sdb, sdc, sdd,** and **sde** to create the Btrfs RAID in the RAID-1C4 configuration.

So, I can mount the Btrfs filesystem **data** in the **/data** directory using the HDD **sdb**, as follows:

```
$ sudo mount /dev/sdb /data
```

As you can see, the Btrfs RAID is mounted in the **/data**

```
$ sudo df -h /data
```

To find the filesystem usage information of the **data** Btrfs filesystem mounted in the **/data**

```
$ sudo btrfs filesystem usage /data
```

As you can see,

The RAID size (**Device size**) is **80 GB** (4×20 GB per HDD).

About **18.99 GB** (**Free (estimated)**) of **80 GB** of disk space can be used in the RAID-1C4 configuration.

In the RAID-1C4 configuration, four copies of the data (**Data ratio**) and four copies of the metadata (**Metadata ratio**) will be stored in the Btrfs filesystem.

As the Btrfs RAID is working, you can unmount it from the **/data** directory, as follows:

```
$ sudo umount /data
```

Setting Up RAID-10

In this section, you will learn how to set up a Btrfs RAID in the RAID-10 configuration using the four HDDs **sdb, sdc, sdd,** and **sde**. The HDDs are 20 GB in size.

```
$ sudo lsblk -e7
```

To create a Btrfs RAID in the RAID-10 configuration using the four HDDs **sdb, sdc, sdd,** and **sde**, run the following command:

```
$ sudo mkfs.btrfs -L data -d raid10 -m raid10 -f /dev/sdb /dev/sdc /dev/sdd /dev/sde
```

Here,

- The –**L** option is used to set the filesystem label **data**.
- The –**d** option is used to set the RAID profile **raid10** for the filesystem data.
- The –**m** option is used to set the RAID profile **raid10** for the filesystem metadata.
- The –**f** option is used to force the creation of the Btrfs filesystem, even if any of the HDDs have an existing filesystem.

The Btrfs filesystem **data** in the RAID-10 configuration should now be created, as you can see in the screenshot below.

You can mount the Btrfs RAID using any HDD/SSD you used to create the RAID.

For example, I used the HDDs **sdb, sdc, sdd,** and **sde** to create the Btrfs RAID in the RAID-10 configuration.

So, I can mount the Btrfs filesystem **data** in the **/data** directory using the HDD **sdb**, as follows:

```
$ sudo mount /dev/sdb /data
```

As you can see, the Btrfs RAID is mounted in the **/data** directory.

```
$ sudo df -h /data
```

To find the filesystem usage information of the data Btrfs filesystem mounted in the **/data** directory, run the following command:

```
$ sudo btrfs filesystem usage /data
```

As you can see,

The RAID size (**Device size**) is **80 GB** (4×20 GB per HDD).

About **39.48 GB** (**Free (estimated)**) of **80 GB** of disk space can be used in the RAID-10 configuration.

In the RAID-10 configuration, two copies of the data (**Data ratio**) and two copies of the metadata (**Metadata ratio**) will be stored in the Btrfs filesystem.

As the Btrfs RAID is working, you can unmount it from the **/data** directory, as follows:

```
$ sudo umount /data
```

Setting Up RAID-5

In this section, you will learn how to set up a Btrfs RAID in the RAID-5 configuration using the four HDDs **sdb, sdc, sdd,** and **sde**. The HDDs are 20 GB in size.

```
$ sudo lsblk
```

To create a Btrfs RAID in the RAID-5 configuration using the four HDDs **sdb, sdc, sdd,** and **sde**, run the following command:

```
$ sudo mkfs.btrfs -L data -d raid5 -m raid5 -f /dev/sdb /dev/sdc /dev/sdd /dev/sde
```

Here,

- The –**L** option is used to set the filesystem label **data**.
- The –**d** option is used to set the RAID profile **raid5** for the filesystem data.
- The –**m** option is used to set the RAID profile **raid5** for the filesystem metadata.
- The –**f** option is used to force the creation of the Btrfs filesystem, even if any of the HDDs have an existing filesystem.

The Btrfs filesystem **data** in the RAID-5 configuration should now be created, as you can see in the screenshot below.

You can mount the Btrfs RAID using any HDD/SSD you used to create the RAID.

For example, I used the HDDs **sdb, sdc, sdd**, and **sde** to create the Btrfs RAID in the RAID-5 configuration.

So, I can mount the Btrfs filesystem **data** in the **/data** directory using the HDD **sdb**, as follows:

```
$ sudo mount /dev/sdb /data
```

As you can see, the Btrfs RAID is mounted in the **/data** directory.

```
$ sudo df -h /data
```

To find the filesystem usage information of the data Btrfs filesystem mounted in the **/data** directory, run the following command:

```
$ sudo btrfs filesystem usage /data
```

As you can see,

The RAID size (**Device size**) is **80 GB** (4×20 GB per HDD).

About **59.24 GB** (**Free (estimated)**) of **80 GB** of disk space can be used in the RAID-5 configuration.

In the RAID-5 configuration, 1.33 copies of the data (**Data ratio**) and 1.33 copies of the metadata (**Metadata ratio**) will be stored in the Btrfs filesystem.

As the Btrfs RAID is working, you can unmount it from the **/data** directory, as follows:

```
$ sudo umount /data
```

Setting Up RAID-6

In this section, you will learn how to set up a Btrfs RAID in the RAID-6 configuration using the four HDDs **sdb, sdc, sdd,** and **sde**. The HDDs are 20 GB in size.

```
$ sudo lsblk
```

To create a Btrfs RAID in the RAID-6 configuration using the four HDDs **sdb, sdc, sdd,** and **sde**, run the following command:

```
$ sudo mkfs.btrfs -L data -d raid6 -m raid6 -f /dev/sdb /dev/sdc /dev/sdd /dev/sde
```

Here,

- The –**L** option is used to set the filesystem label **data**.
- The –**d** option is used to set the RAID profile **raid6** for the filesystem data.
- The –**m** option is used to set the RAID profile **raid6** for the filesystem metadata.
- The –**f** option is used to force the creation of the Btrfs filesystem, even if any of the HDDs have an existing filesystem.

The Btrfs filesystem **data** in the RAID-6 configuration should now be created, as you can see in the screenshot below.

You can mount the Btrfs RAID using any HDD/SSD you used to create the RAID.

For example, I used the HDDs **sdb, sdc, sdd,** and **sde** to create the Btrfs RAID in the RAID-6 configuration.

So, I can mount the Btrfs filesystem **data** in the **/data** directory using the HDD **sdb**, as follows:

```
$ sudo mount /dev/sdb /data
```

As you can see, the Btrfs RAID is mounted in the **/data** directory.

```
$ sudo df -h /data
```

To find the filesystem usage information of the **data** Btrfs filesystem mounted in the **/data** directory, run the following command:

```
$ sudo btrfs filesystem usage /data
```

As you can see,

The RAID size (**Device size**) is **80 GB** (4×20 GB per HDD).

About **39.48 GB** (**Free (estimated)**) of **80 GB** of disk space can be used in the RAID-6 configuration.

In the RAID-6 configuration, two copies of the data (**Data ratio**) and two copies of the metadata (**Metadata ratio**) will be stored in the Btrfs filesystem.

As the Btrfs RAID is working, you can unmount it from the **/data** directory, as follows:

```
$ sudo umount /data
```

Problems with Btrfs RAID-5 and RAID-6

The built-in Btrfs RAID-5 and RAID-6 configurations are still experimental. These configurations are very unstable and you should not use them in production. BETTER TO NOT USE 5 and 6

To prevent data corruption, the Ubuntu operating system did not implement RAID-5 and RAID-6 for the Btrfs filesystem. So, you will not be able to create a Btrfs RAID in the RAID-5 and RAID-6 configurations using the built-in RAID feature of the Btrfs filesystem on Ubuntu. That is why I have shown you how to create a Btrfs RAID in the RAID-5 and RAID-6 configurations in Fedora 33, instead of Ubuntu 20.04 LTS.

Mounting a Btrfs RAID Automatically on Boot

To mount a Btrfs RAID automatically at boot time using the /etc/fstab file, you will need to know the UUID of the Btrfs filesystem.

You can find the UUID of a Btrfs filesystem with the following command:

```
$ sudo blkid --match-token TYPE=btrfs
```

As you can see, the UUID of the storage devices that are added to the Btrfs filesystem for configuring the RAID is the same.

In my case, it is **c69a889a-8fd2-4571-bd97-a3c2e4543b6b**. It will be different for you. So, be sure to replace this UUID with yours from now on.

Now, open the **/etc/fstab** file with the nano text editor, as follows:

```
$ sudo nano /etc/fstab
```

Add the following line to the end of the **/etc/fstab** file.

```
UUID=<strong>c69a889a-8fd2-4571-bd97-a3c2e4543b6b</strong>  /data   btrfs   defaults    0   0
```

Once you are finished, press **<Ctrl> + X** followed by **Y** and **<Enter>** to save the **/etc/fstab** file.

For the changes to take effect, restart your computer, as follows:

```
$ sudo reboot
```

As you can see, the Btrfs RAID is correctly mounted in the **/data** directory.

```
$ df -h /data
```

As you can see, the Btrfs RAID mounted in the **/data** directory is working just fine.

```
$ sudo btrfs filesystem usage /data
```